

RESTful API Design Standards

Comprehensive Architecture & Best Practices Guide for Backend Engineers

- PRODUCTION STANDARD
- REST GUIDELINES

1. Introduction & Core Architecture

A **REST (Representational State Transfer) API** acts as the **standardized interface between backend servers and clients** (Frontend web apps, mobile applications, or external services). Adhering to strict architectural standards ensures scalability, maintainability, and seamless developer experience across engineering teams.

2. Endpoint Naming Conventions

Endpoints should strictly represent **resources (nouns)** rather than actions (verbs). Follow these industry standards:

- Use Plural Nouns:** Collections must be named in plural form.

✓ GET /api/v1/users

✗ GET /api/v1/getUser or /api/v1/user
- Use Kebab-case (Hyphens):** Separate multi-word resources with hyphens for readability.

✓ GET /api/v1/order-items

✗ GET /api/v1/orderItems or /order_items
- Hierarchical Relationships (Nested Resources):** Express parent-child relations clearly.

✓ GET /api/v1/users/12/orders (Retrieves orders specifically belonging to User 12).

3. HTTP Methods & Idempotency

HTTP Method	Primary Use Case	Idempotent?
GET	Retrieve data (read-only) without modifying server state.	Yes
POST	Create a new resource on the server.	No
PUT	Replace an existing resource entirely (Full Update).	Yes
PATCH	Apply partial modifications to a resource (Partial Update).	No
DELETE	Remove a specified resource by its unique identifier.	Yes

4. Standardized Response Envelope

All API responses must maintain a consistent JSON structure to streamline client-side handling:

Success Envelope

```
{
  "success": true,
  "message": "Resource retrieved successfully",
  "data": {
    "id": "usr_9912",
    "name": "Alex Johnson",
    "email": "alex@example.com"
  }
}
```

Error Envelope

```
{
  "success": false,
  "message": "Validation Failed",
  "errors": [
    {
      "field": "email",
      "message": "Invalid email address format."
    }
  ]
}
```

5. HTTP Status Codes Guidelines

- **200 OK** : Successful request returning requested payload.
- **201 Created** : Resource created successfully (return created object payload).
- **204 No Content** : Successful execution (e.g., deletion) with no returning payload body.
- **400 Bad Request** : Invalid payload, syntax error, or client-side validation failure.
- **401 Unauthorized** : Missing or invalid authentication credentials (Unauthenticated).
- **403 Forbidden** : Authenticated user lacks sufficient permissions (Unauthorized access).
- **404 Not Found** : The requested resource endpoint or entity does not exist.
- **422 Unprocessable Entity** : Syntax is correct, but failed domain validation rules.
- **500 Internal Server Error** : Unexpected backend exception or server crash.

6. Query Parameters: Search, Filtering & Pagination

Do not create dedicated endpoints for filtering or pagination. Consolidate them into query parameters on the collection route:

```
GET /api/v1/products?search=laptop&category=electronics&sortBy=price&order=asc&page=1&limit=10
```

Standard Paginated Response Payload:

```
{
  "success": true,
  "data": [ /* Array of item objects */ ],
  "pagination": {
    "totalItems": 150,
    "currentPage": 1,
    "totalPages": 15,
    "limit": 10
  }
}
```

7. Versioning, Security & Documentation

- **API Versioning:** Always prefix routes with version identifiers (e.g., `/api/v1/`) to prevent breaking existing mobile/web clients when introducing breaking architectural updates.
- **Security Best Practices:**
 - Enforce HTTPS strictly for data encryption in transit.
 - Pass authentication tokens via standard HTTP headers: `Authorization: Bearer <token>` .
 - Implement Rate Limiting middleware to prevent DDoS and Brute-Force attacks.
- **API Documentation:** Generate dynamic, interactive documentation using **OpenAPI (Swagger)** to provide front-end engineers with clear schema contracts and testing environments.